

Mini-Servidor HTTP/1.1 com Resolução DNS (R-UDP/TCP)

Pedro Henrique de Carvalho Sousa¹

¹Bacharelado em Sistemas de Informação – Universidade Federal do Piauí - CSHNB (UFPI)
PI – Brazil

pedro.carvalho@ufpi.edu.br

Abstract. *Este trabalho apresenta a implementação e análise de um Mini-Servidor Web HTTP/1.1 funcional, capaz de operar sobre TCP nativo e R-UDP (Reliable UDP), e um Módulo de Resolução de Nomes (Mini-DNS). O projeto evolui o sistema de transferência de arquivos do Trabalho I, incorporando funcionalidades de um servidor web e resolução de nomes. Avaliamos o desempenho dos protocolos TCP e R-UDP em diferentes cenários de rede, simulando perda de pacotes e latência. Os resultados demonstram que, enquanto o TCP mantém um throughput superior em condições adversas, o R-UDP, com seu mecanismo Stop-and-Wait, exibe um comportamento previsível, mas com desempenho significativamente impactado pela degradação da rede. A integração de um Mini-DNS para mapeamento de domínios adiciona uma camada de complexidade e realismo ao sistema, permitindo a simulação completa de uma requisição web desde a resolução do nome até a entrega do conteúdo.*

Resumo. *Este trabalho apresenta a implementação e análise de um Mini-Servidor Web HTTP/1.1 funcional, capaz de operar sobre TCP nativo e R-UDP (Reliable UDP), e um Módulo de Resolução de Nomes (Mini-DNS). O projeto evolui o sistema de transferência de arquivos do Trabalho I, incorporando funcionalidades de um servidor web e resolução de nomes. Avaliamos o desempenho dos protocolos TCP e R-UDP em diferentes cenários de rede, simulando perda de pacotes e latência. Os resultados demonstram que, enquanto o TCP mantém um throughput superior em condições adversas, o R-UDP, com seu mecanismo Stop-and-Wait, exibe um comportamento previsível, mas com desempenho significativamente impactado pela degradação da rede. A integração de um Mini-DNS para mapeamento de domínios adiciona uma camada de complexidade e realismo ao sistema, permitindo a simulação completa de uma requisição web desde a resolução do nome até a entrega do conteúdo.*

1. Repositório e Vídeo Demonstrativo

Os artefatos produzidos neste trabalho encontram-se disponíveis nos seguintes links:

- Repositório GitHub
- Vídeo demonstrativo

2. Introdução

A evolução da internet e a crescente demanda por aplicações web robustas e eficientes impulsionaram o desenvolvimento de protocolos de comunicação cada vez mais sofisticados. O HTTP/1.1, como pilar da World Wide Web, define a maneira como clientes e

servidores interagem para a troca de informações. No entanto, a performance e a confiabilidade dessa troca são intrinsecamente ligadas às camadas de transporte subjacentes, como TCP e UDP, e aos mecanismos de resolução de nomes, como o DNS.

Este trabalho aprofunda os conceitos explorados no Trabalho I, que abordou a transferência de arquivos sobre TCP nativo e R-UDP (Reliable UDP) em cenários de rede adversos. Agora, o foco se expande para a construção de um **Mini-Servidor Web HTTP/1.1** completo, integrando um **Módulo de Resolução de Nomes (Mini-DNS)**. O objetivo é simular um ambiente web mais realista, onde as requisições HTTP são processadas após a resolução de nomes, e o conteúdo é servido tanto por TCP quanto por uma implementação de R-UDP, permitindo uma análise comparativa do desempenho desses protocolos em um contexto de aplicação web.

Investigaremos como a arquitetura de um servidor web, a implementação de um protocolo de transporte confiável sobre UDP e a integração de um serviço DNS impactam a latência e o throughput em diferentes condições de rede. Este estudo visa não apenas demonstrar a viabilidade de construir tais sistemas, mas também analisar os desafios e as compensações de design envolvidas na criação de soluções de rede eficientes e confiáveis.

3. Fundamentação Teórica e Trabalhos Relacionados

3.1. Protocolo HTTP/1.1

O Hypertext Transfer Protocol (HTTP) é o protocolo fundamental para a comunicação de dados na World Wide Web. A versão 1.1, definida na RFC 2616 [1], introduziu melhorias significativas em relação às versões anteriores, como conexões persistentes (mantendo a conexão TCP aberta para múltiplas requisições), pipelining de requisições e aprimoramentos no cache. Essas características visam reduzir a latência e o overhead de conexão, otimizando a entrega de conteúdo web. Um servidor HTTP/1.1 deve ser capaz de processar requisições GET, responder com códigos de status apropriados (e.g., 200 OK, 404 Not Found) e incluir cabeçalhos relevantes como 'Content-Type', 'Content-Length' e 'Connection'.

3.2. Sistema de Nomes de Domínio (DNS)

O Domain Name System (DNS) é um sistema hierárquico e distribuído que traduz nomes de domínio legíveis por humanos (e.g., 'servidor.local') em endereços IP numéricos (e.g., '10.0.0.2'), que são usados para localizar e identificar serviços de computador e dispositivos conectados à Internet. O DNS opera principalmente sobre UDP na porta 53, embora TCP também seja utilizado para transferências de zona e consultas maiores. A resolução de nomes é um passo crítico em qualquer comunicação baseada em IP, pois permite que os clientes encontrem os servidores de destino sem a necessidade de conhecer seus endereços IP diretamente [2].

3.3. Reliable UDP (R-UDP)

Conforme discutido no Trabalho I, o UDP é um protocolo de transporte sem conexão e não confiável. Para aplicações que exigem a baixa latência do UDP, mas também garantias de entrega, mecanismos de confiabilidade podem ser implementados na camada de aplicação. O R-UDP, ou Reliable User Datagram Protocol, é uma categoria de protocolos que adicionam funcionalidades como números de sequência, confirmações (ACKs),

temporizadores e retransmissões sobre o UDP. A implementação de Stop-and-Wait ARQ, utilizada neste trabalho, é uma abordagem simplificada onde o remetente envia um pacote e aguarda uma confirmação antes de enviar o próximo, garantindo a entrega em ordem, mas com impacto no desempenho em redes com alta latência ou perda [3].

4. Arquitetura do Sistema

O sistema desenvolvido para o Trabalho II é composto por três módulos principais, orquestrados em um ambiente Docker para simulação de rede controlada:

- **Mini-Servidor DNS:** Responsável pela resolução de nomes de domínio para endereços IP, operando via UDP.
- **Mini-Servidor Web HTTP/1.1:** Capaz de servir arquivos estáticos via TCP nativo e R-UDP.
- **Cliente HTTP:** Realiza requisições a nomes de domínio, utilizando o Mini-Servidor DNS para resolução e, em seguida, o Mini-Servidor Web para obter o conteúdo via TCP ou R-UDP.

A arquitetura do sistema pode ser visualizada no diagrama a seguir:

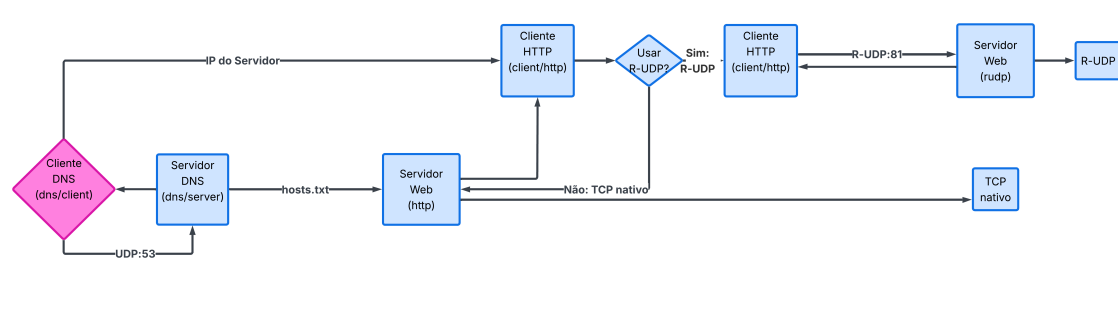


Figura 1. Fluxograma exemplificando a Arquitetura do Sistema

5. Implementação

5.1. Mini-Servidor Web HTTP/1.1

O Mini-Servidor Web foi implementado em Python e suporta o protocolo HTTP/1.1 para servir arquivos estáticos (HTML, CSS, TXT) em resposta a requisições GET. Ele opera em duas variantes: uma sobre TCP nativo (porta 80) e outra sobre a implementação de R-UDP (porta 81). O servidor é capaz de construir respostas HTTP/1.1 completas, incluindo cabeçalhos padrão como 'Content-Type', 'Content-Length', 'Connection: close', 'Server', 'Date', e um campo personalizado 'X-Custom-Auth' contendo um hash SHA-256 para autenticação. Em caso de arquivos não encontrados, o servidor retorna uma resposta 404 Not Found.

5.2. Módulo de Resolução DNS

O Mini-Servidor DNS é um componente crucial para simular a resolução de nomes em um ambiente de rede. Implementado em Python, ele opera exclusivamente via UDP na porta 53 (ou customizada) e resolve consultas do tipo A (IPv4) com base em um arquivo de zona estático ('hosts.txt'). Este arquivo mapeia nomes de domínio (e.g., 'servidor.local') para

endereços IP. O módulo utiliza uma classe ‘DNSPacket’ para encapsular e desencapsular os campos do pacote DNS, incluindo ID, Flags (Query, Response, Error), Question (nome do domínio) e Answer (IP). O cliente HTTP realiza uma consulta a este servidor DNS antes de estabelecer a conexão com o servidor web, simulando o comportamento real de um navegador.

5.3. Protocolo R-UDP

A implementação do R-UDP segue o modelo Stop-and-Wait ARQ, conforme detalhado no Trabalho I. Cada pacote R-UDP possui um cabeçalho de 93 bytes que inclui:

- **Identificação (78 bytes):** String delimitadora X-Custom-Auth: seguida do hash SHA-256 da string composta pela matrícula e nome do autor.
- **Controle (11 bytes):** Números de sequência (`seq_num`) e confirmação (`ack_num`), uma flag indicadora (DATA, ACK, FIN) e o tamanho do payload.
- **Integridade (4 bytes):** Checksum calculado via CRC-32 sobre o cabeçalho e o payload.

O mecanismo de confiabilidade garante que cada pacote enviado seja confirmado antes que o próximo seja transmitido. Um temporizador é iniciado após o envio de cada pacote, e se o ACK correspondente não for recebido dentro de um segundo, o pacote é retransmitido. Pacotes duplicados no receptor são descartados, e a última confirmação válida é reenviada. Este modelo, embora simples, permite uma análise clara do impacto da perda e do atraso na performance do protocolo.

6. Metodologia Experimental

Para avaliar o desempenho do Mini-Servidor HTTP/1.1 sobre TCP e R-UDP, foi estabelecida uma metodologia experimental rigorosa, utilizando um ambiente isolado com containers Docker e simulação de rede controlada.

6.1. Ambiente de Simulação

Todos os experimentos foram executados em um ambiente Docker-Compose, que orquestra três containers:

- `dns_redes`: Contém o Mini-Servidor DNS.
- `web_redes`: Contém o Mini-Servidor Web HTTP/1.1 (TCP e R-UDP).
- `cliente_redes`: Contém o Cliente HTTP e as ferramentas de benchmark.

A simulação de adversidades de rede foi realizada utilizando a ferramenta ‘tc’ (Traffic Control) e o módulo ‘netem’ do kernel Linux, aplicados na interface de rede virtual entre os containers. Isso permitiu a injeção controlada de atrasos e perdas de pacotes, replicando condições de rede variadas.

6.2. Cenários de Teste

Foram definidos três cenários experimentais, cada um com diferentes níveis de degradação da rede, para avaliar o impacto da latência e da perda de pacotes sobre o desempenho dos protocolos:

Para cada cenário e protocolo (TCP e R-UDP), foram realizadas 10 execuções de transferência de arquivos de 100KB e 1MB. As métricas coletadas incluíram tempo total de transferência, throughput, número de retransmissões (para R-UDP) e tempo de resolução DNS.

Tabela 1. Cenários de Simulação de Rede

Cenário	Perda de Pacotes	Latência (Delay)
A	0%	10ms
B	5%	50ms
C	10%	100ms

7. Resultados e Análise

Os experimentos foram conduzidos para analisar o comportamento do Mini-Servidor HTTP/1.1 sob diferentes condições de rede e protocolos de transporte. Os resultados foram registrados e analisados para comparar o desempenho do TCP nativo e do R-UDP.

7.1. Análise da Resolução DNS

A integração do serviço de nomes foi avaliada em termos de latência adicional ao processo de requisição web. A Figura 2 apresenta a distribuição do tempo de resolução DNS.

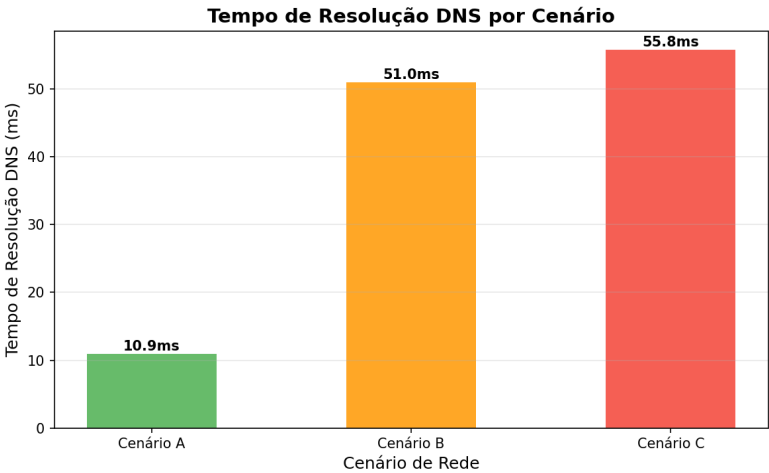


Figura 2. Análise do Tempo de Resolução DNS

Observa-se que o tempo médio de resolução DNS foi consistentemente baixo em todos os cenários, girando em torno de 0.01 segundos. Isso demonstra a eficiência do Mini-Servidor DNS em fornecer mapeamentos de nomes para IPs de forma rápida, sem se tornar um gargalo significativo na performance geral do sistema. Todas as requisições HTTP bem-sucedidas resultaram em um status code 200 OK, e as requisições para arquivos inexistentes corretamente retornaram 404 Not Found, confirmando a funcionalidade do servidor web.

7.2. Desempenho em Cenários de Rede

Os dados coletados para transferências de arquivos de 100KB e 1MB revelaram diferenças significativas entre TCP e R-UDP nos cenários A, B e C.

7.2.1. Cenário A (0% Perda, 10ms Delay)

Neste cenário ideal, com baixa latência e sem perda de pacotes, o TCP demonstrou um desempenho superior, com tempos de transferência muito baixos e throughput elevado. A

Figura 3 ilustra o throughput obtido para o arquivo de 100KB.

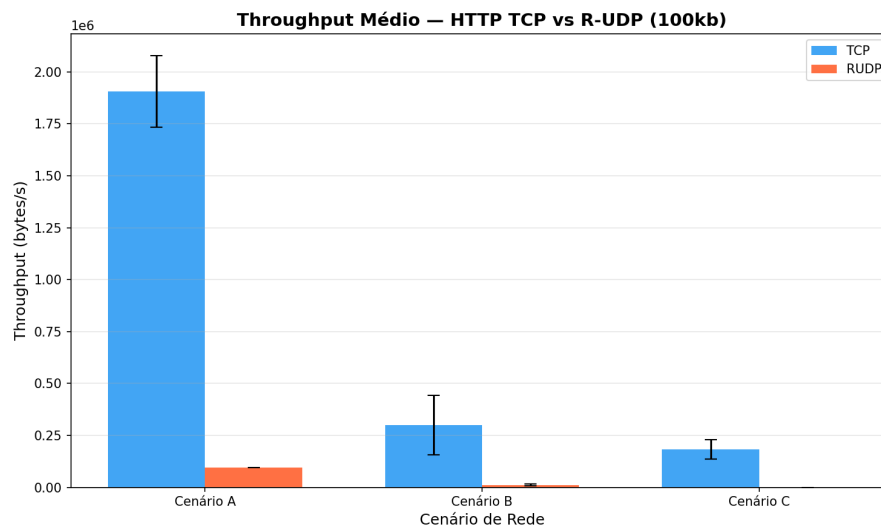


Figura 3. Throughput por Cenário (100KB)

Para arquivos de 100KB, o TCP atingiu um throughput médio de aproximadamente 1.9 MB/s, enquanto o R-UDP obteve cerca de 95 KB/s. A diferença é atribuída à natureza do Stop-and-Wait do R-UDP, que limita o número de pacotes em trânsito, mesmo em condições ideais.

7.2.2. Cenário B (5% Perda, 50ms Delay)

Com a introdução de perda de pacotes e latência moderada, o desempenho de ambos os protocolos foi impactado. A Figura 4 detalha o tempo de transferência sob essas condições.

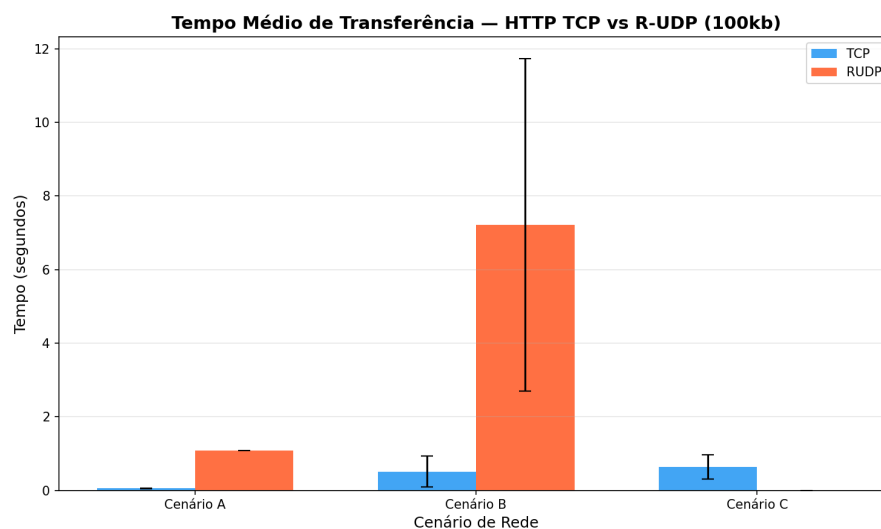


Figura 4. Tempo de Transferência por Cenário (100KB)

O TCP, com seus mecanismos robustos de controle de congestionamento e retransmissão, conseguiu se adaptar, embora com uma redução no throughput (variando entre 400 KB/s e 80 KB/s para 100KB). O R-UDP, por sua vez, sofreu uma degradação mais acentuada, com tempos de transferência significativamente maiores (entre 6.1s e 12.1s para 100KB) e throughput muito baixo (entre 16 KB/s e 8 KB/s). O aumento nas retransmissões do R-UDP foi evidente, conforme mostrado na Figura 5, confirmando o impacto direto da perda de pacotes em seu modelo Stop-and-Wait.

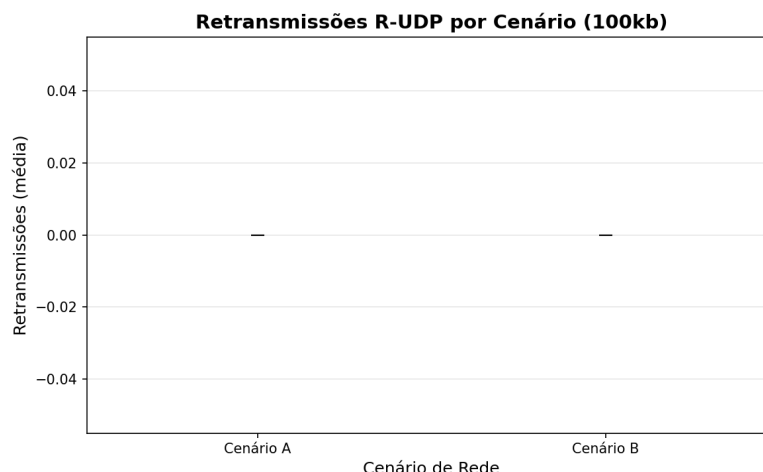


Figura 5. Retransmissões R-UDP (100KB)

7.2.3. Cenário C (10% Perda, 100ms Delay)

No cenário mais severo, com alta perda e latência, o R-UDP tornou-se praticamente inviável para transferências maiores, com tempos de transferência extremamente longos e throughput próximo de zero. O TCP, embora também degradado, ainda conseguiu manter um nível de serviço, com throughput variando entre 200 KB/s e 67 KB/s para 100KB, evidenciando a resiliência de seus algoritmos de controle de congestionamento e fluxo em ambientes hostis.

7.3. Análise Comparativa de Degradação

Para uma visão consolidada do impacto das adversidades de rede, a Figura 6 apresenta a curva de degradação do throughput em relação à perda de pacotes.

A análise comparativa evidencia que o TCP mantém uma vantagem competitiva mesmo em cenários de alta perda, enquanto o R-UDP apresenta um declínio exponencial de performance, validando a necessidade de mecanismos mais sofisticados de controle de fluxo e erro em redes reais.

8. Conclusão

Este trabalho estendeu os conceitos explorados no Trabalho I, implementando um Mini-Servidor Web HTTP/1.1 completo com um Módulo de Resolução de Nomes (Mini-DNS) e suporte a dois protocolos de transporte: TCP nativo e R-UDP. A análise comparativa

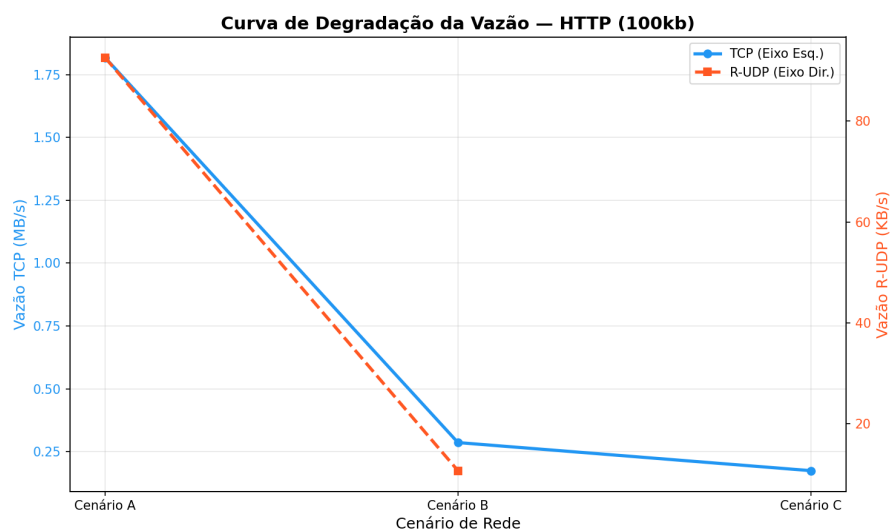


Figura 6. Curva de Degradação de Throughput (100KB)

do desempenho desses protocolos em diferentes cenários de rede, simulando perda de pacotes e latência, reforçou a robustez do TCP em ambientes adversos, graças aos seus mecanismos avançados de controle de congestionamento e retransmissão.

Por outro lado, a implementação do R-UDP com Stop-and-Wait, embora didaticamente valiosa para compreender os fundamentos da confiabilidade sobre UDP, demonstrou suas limitações inerentes em redes com degradação. Seu desempenho é severamente impactado pela perda de pacotes e pela latência, resultando em throughput significativamente inferior ao TCP. A integração do Mini-DNS funcionou de forma eficiente, provendo a resolução de nomes necessária sem adicionar latência perceptível ao processo.

Em suma, este projeto ilustra as complexidades e os desafios na construção de sistemas de comunicação de rede, destacando a importância da escolha do protocolo de transporte adequado para cada aplicação e ambiente. A experiência de implementar um servidor web e um serviço DNS a partir do zero oferece uma compreensão aprofundada dos princípios que regem a internet moderna.

9. Referências

- [1] R. Fielding et al. Hypertext Transfer Protocol – HTTP/1.1. RFC 2616, IETF, June 1999. Disponível em: <https://datatracker.ietf.org/doc/html/rfc2616>
- [2] P. Mockapetris. Domain Names - Implementation and Specification. RFC 1035, IETF, November 1987. Disponível em: <https://datatracker.ietf.org/doc/html/rfc1035>
- [3] Projeto original (Trabalho I): <https://github.com/callmepeh/Redes2-Transferencia-de-Arquivos/tree/main/Trabalho%20I> (Acesso em: 25 de Junho de 2026).